



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**

Membre de
HONORIS UNITED UNIVERSITIES

IADATA

**INTELLIGENCE
ARTIFICIELLE
& SCIENCES
DES DONNEES**

ACADEMIC YEAR

2025/2023

APPRENTISSAGE FÉDÉRÉ

THEME

**Conception et implémentation d'un systém
distribué**

DEVOLEPED BY

Hafsa sabri

**Fatima Ezzahra
Elhadrachi**

Rania El harch

Soufiane Saddoun

Table des matières

1. Introduction	1
1.1. Contexte	1
1.2. Cas d'usage réels	2
1.3. Objectifs du projet	3
2. Architecture du Système	4
2.1. Vue d'ensemble.....	4
2.2. Composants logiciels	5
2.3. Choix technologiques	6
3. Partie Théorique.....	7
3.1. Algorithme FedAvg.....	7
3.2. Distribution IID vs Non-IID.....	8
3.3. Synchronisation : Horloges de Lamport	9
3.4. Élection de leader : Algorithme Bully.....	10
3.5. Théorème CAP et compromis	11
3.6. Sécurité Byzantine et algorithme Krum	12
4. Implémentation	13
4.1. Protocole de communication	13
4.2. Cycle de vie d'une ronde	14
4.3. Gestion des pannes	15
4.4. Conteneurisation Docker	16
5. Partie Pratique : Guide d'Utilisation et Résultats	17
5.1. Installation et prérequis	17
5.2. Exécution locale	17
5.3. Exécution avec Docker	18
5.4. Tests de robustesse.....	19
5.4.1 Scénario : Crash de client	19
5.4.2 Scénario : Attaque Byzantine.....	20
5.4.3 Scénario : Détection de mises à jour obsolètes.....	20
5.5. Validation des horloges de Lamport	21
5.6. Élection de leader (Bully)	21
6. Analyse et Discussion	22
6.1. Forces du système.....	22

6.2. Limitations.....	22
6.3. Améliorations possibles	23
6.4. Gestion de projet	23
7. Conclusion.....	24

8. Références

1. Introduction

1.1 Contexte

L'apprentissage automatique traditionnel exige la centralisation de toutes les données d'entraînement sur un serveur unique. Cette approche pose deux problèmes majeurs : la confidentialité (les données sensibles quittent l'environnement de l'utilisateur) et la scalabilité (un serveur central devient rapidement un goulot d'étranglement).

L'apprentissage fédéré (Federated Learning, FL), introduit par McMahan et al. en 2017 chez Google, propose une solution radicale : entraîner un modèle global sans jamais transférer les données brutes. Chaque client conserve ses données localement, entraîne un modèle sur ses propres exemples, et n'envoie que les paramètres mis à jour (gradients ou poids) à un serveur d'agrégation. Le serveur combine ces contributions et redistribue le modèle global amélioré.

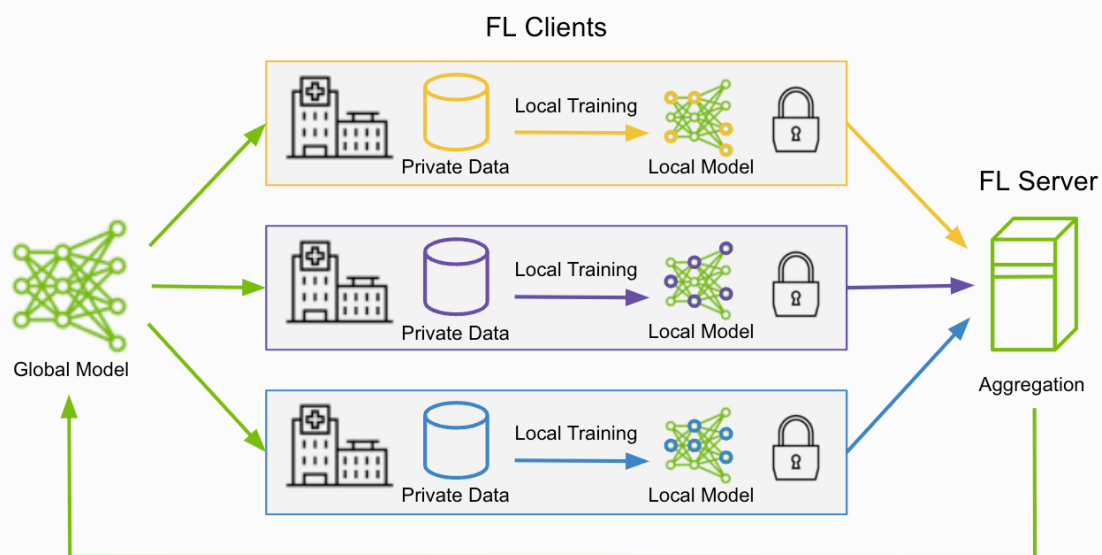


Figure 1.1 — Principe général de l'apprentissage fédéré (source : Google AI Blog)

1.2 Cas d'Usage Réels

L'apprentissage fédéré est aujourd'hui déployé dans plusieurs secteurs critiques :

- **Claviers prédictifs (Gboard)** : Google entraîne le modèle de prédiction de texte directement sur les téléphones des utilisateurs sans jamais collecter leurs frappes.
- **Santé** : des hôpitaux collaborent sur un modèle de diagnostic sans échanger les dossiers patients, respectant ainsi les réglementations comme le RGPD.
- **Finance** : des banques détectent la fraude collectivement sans partager les transactions de leurs clients.
- **Véhicules autonomes** : les voitures partagent leur expérience de conduite via des modèles, pas via leurs trajets.

1.3 Objectifs du Projet

Ce projet vise à concevoir et implémenter un système d'apprentissage fédéré complet en mettant l'accent sur les défis distribués :

- Architecture distribuée client-serveur avec coordination centrale (puis multi-leader avec élection).
- Gestion de la cohérence des données et synchronisation des rondes d'entraînement.
- Tolérance aux pannes : crashes de clients, partitions réseau, messages perdus, attaques byzantines.
- Synchronisation logique via les horloges de Lamport.
- Élection de leader automatique via l'algorithme Bully.
- Déploiement reproductible via Docker et docker-compose.

2. Architecture du Système

2.1 Vue d'Ensemble

Notre système suit une architecture client-serveur distribuée. Un coordinateur central (ou plusieurs, en mode multi-leader) orchestre les rondes d'apprentissage, tandis que N clients entraînent localement leur copie du modèle global sur des partitions distinctes du jeu de données MNIST.

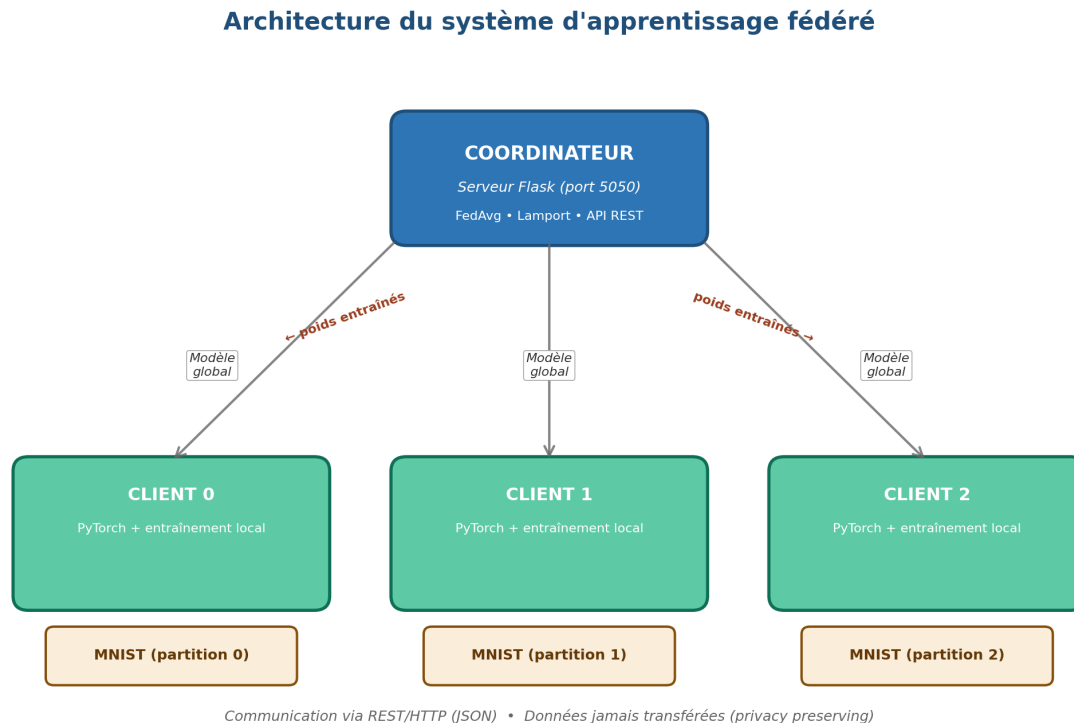


Figure 2.1 — Architecture client-serveur du système d'apprentissage fédéré

2.2 Composants Logiciels

Le système est implémenté en Python 3.11 avec les composants suivants :

Composant	Fichier	Rôle
Modèle CNN	model/cnn.py	Réseau de neurones convolutionnel pour MNIST (215K paramètres)
Horloge Lamport	model/lamport_clock.py	Implémentation des horloges logiques de Lamport
Partitionneur	data/partition.py	Partitionnement IID et non-IID des données
Agrégateur	server/aggregator.py	Algorithmes FedAvg et Krum (Byzantine-tolerant)

Composant	Fichier	Rôle
Coordinateur	server/coordinator.py	Serveur Flask gérant les rondes d'apprentissage
Élection Bully	server/bully.py	Algorithme d'élection de leader
Client FL	client/client.py	Boucle d'entraînement local et communication
Injecteur de pannes	client/failure_injector.py	Simulation de défaillances (5 modes)

2.3 Choix Technologiques

Plusieurs choix techniques ont été faits pour équilibrer simplicité, performances et compatibilité :

- **PyTorch (CPU-only)** : framework de deep learning. La version CPU-only (~150 Mo) est utilisée car le projet vise à démontrer les aspects distribués, pas la performance brute du GPU.
- **Flask + REST/JSON** : communication HTTP simple à déboguer. gRPC aurait été plus performant mais plus complexe à mettre en œuvre dans le temps imparti.
- **Docker + docker-compose** : déploiement reproductible et isolation des composants. Trois configurations sont fournies : normale, simulation de pannes, et multi-coordonateur Bully.
- **MNIST** : jeu de données classique de chiffres manuscrits (60 000 images d'entraînement, 10 000 de test). Suffisamment petit pour entraîner rapidement, suffisamment riche pour observer les effets distribués.

3. Partie Théorique

Cette section justifie l'architecture choisie et décrit les algorithmes distribués mis en œuvre, en analysant les compromis fondamentaux.

3.1 Algorithme FedAvg

FedAvg (Federated Averaging), proposé par McMahan et al. en 2017, est l'algorithme fondateur de l'apprentissage fédéré. Sa formule est simple mais cruciale :

$$w_{global} = \sum_k (n_k / n_{total}) \times w_k$$

Où w_k représente les poids reçus du client k , n_k le nombre d'échantillons utilisés par ce client, et n_{total} la somme totale des échantillons. Les clients ayant entraîné sur plus de données ont plus d'influence — ce qui correspond à l'intuition statistique de la moyenne pondérée.

Notre implémentation se trouve dans `server/aggregator.py` et fonctionne couche par couche : pour chaque couche du réseau (`conv1`, `conv2`, `fc1`, `fc2`), nous calculons la moyenne pondérée des tenseurs de poids reçus.

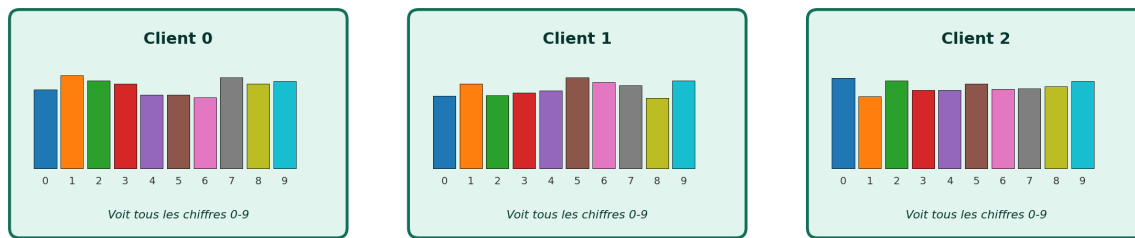
3.2 Distribution IID vs Non-IID

La répartition des données entre clients est un facteur déterminant pour la convergence et la qualité du modèle global :

- **IID (Independent and Identically Distributed)** : chaque client reçoit un échantillon aléatoire uniforme. Tous les clients voient des distributions similaires des 10 chiffres MNIST. Cas idéal pour FedAvg.
- **Non-IID** : chaque client ne voit que 2 ou 3 classes de chiffres. Représente la réalité (un téléphone d'un utilisateur n'a pas un échantillon uniforme du langage humain). Pose un défi majeur car les modèles locaux divergent fortement.

IID — Distribution Identique et Indépendante (cas idéal)

Chaque client reçoit un échantillon aléatoire uniforme des 10 chiffres



Non-IID — Distribution Biaisée (cas réaliste, ex: téléphones, hôpitaux)

Chaque client ne voit que 2 classes différentes — défi majeur pour FedAvg

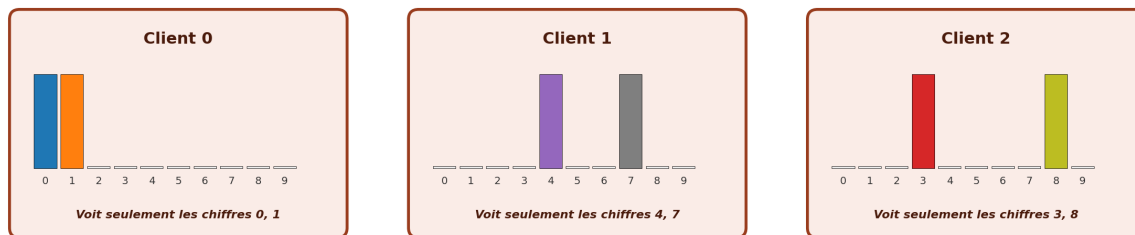


Figure 3.1 — Comparaison de la distribution IID et non-IID des données entre clients

3.3 Synchronisation : Horloges de Lamport

Dans un système distribué, les horloges physiques des différentes machines ne sont pas fiables — elles dérivent et leur synchronisation NTP est coûteuse et imparfaite. Pour ordonner causalement les événements sans dépendre du temps physique, Leslie Lamport a proposé en 1978 les horloges logiques.

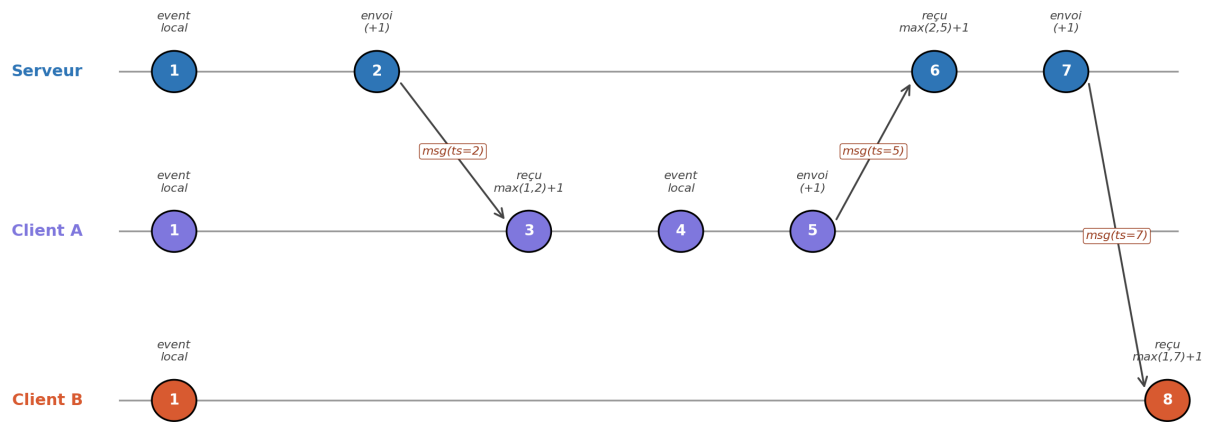
Chaque nœud (serveur ou client) maintient un compteur entier suivant trois règles :

- **Règle 1 (événement local) :** compteur += 1
- **Règle 2 (envoi de message) :** compteur += 1, attacher le compteur au message
- **Règle 3 (réception de message) :** compteur = max(compteur_local, compteur_reçu) + 1

Cette discipline garantit que si un événement A a causalement précédé B, alors $\text{clock}(A) < \text{clock}(B)$. Notre implémentation se trouve dans `model/lamport_clock.py` et est utilisée dans toutes les communications client-serveur.

Horloges Logiques de Lamport — Échange entre 3 nœuds

Règles : événement local $\rightarrow +1$; envoi $\rightarrow +1$, attacher ; réception $\rightarrow \max(\text{local}, \text{reçu}) + 1$



Garantie : si A a causalement précédé B, alors $\text{clock}(A) < \text{clock}(B)$. Le contraire n'est pas vrai (limite des horloges Lamport).

Figure 3.2 — Évolution des horloges logiques lors d'échanges entre 3 nœuds

3.4 Élection de Leader : Algorithme Bully

Notre système initial avec un coordinateur unique présente un point de défaillance unique (single point of failure) : si le coordinateur tombe, tout le système s'arrête. L'algorithme Bully de Garcia-Molina (1982) résout ce problème en permettant à plusieurs coordinateurs d'élire automatiquement un nouveau leader lorsque l'actuel devient indisponible.

Principe : chaque coordinateur a un identifiant unique (entier). Le coordinateur avec le plus grand identifiant doit être leader. L'algorithme repose sur trois types de messages :

- **ELECTION** : envoyé par un nœud qui détecte un leader silencieux, vers tous les nœuds avec un ID supérieur.
- **OK (réponse)** : un nœud avec un ID supérieur répond pour supprimer l'initiateur, puis lance sa propre élection.
- **COORDINATOR** : diffusion par le nouveau leader pour annoncer son couronnement.

Algorithme Bully — Élection après crash du leader

Le coordinateur avec l'ID le plus élevé devient leader. Si le leader meurt, les survivants ré-élisent automatiquement.

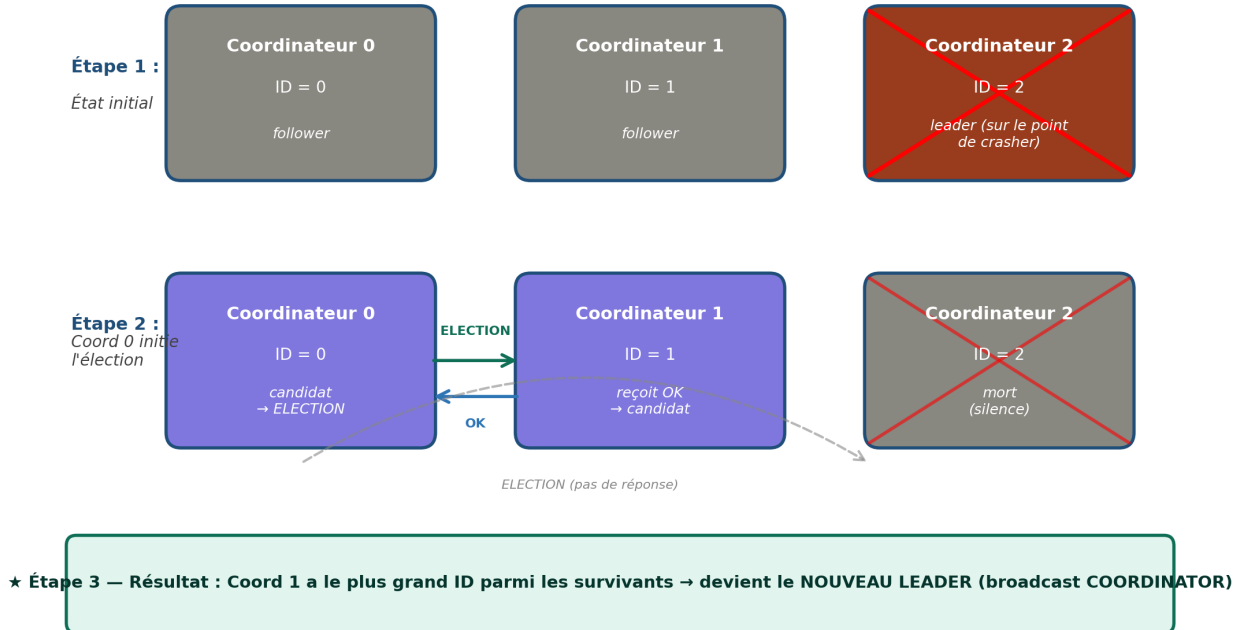


Figure 3.3 — Déroulement type d'une élection Bully après crash du leader

3.5 Théorème CAP et Compromis

Le théorème CAP de Brewer (2000), formellement prouvé par Gilbert et Lynch (2002), stipule qu'un système distribué ne peut garantir simultanément que deux des trois propriétés suivantes :

- **Cohérence (Consistency)** : tous les nœuds voient la même donnée au même moment.
- **Disponibilité (Availability)** : chaque requête reçoit une réponse (succès ou échec).
- **Tolérance au partitionnement (Partition tolerance)** : le système continue de fonctionner malgré des pertes de messages réseau.

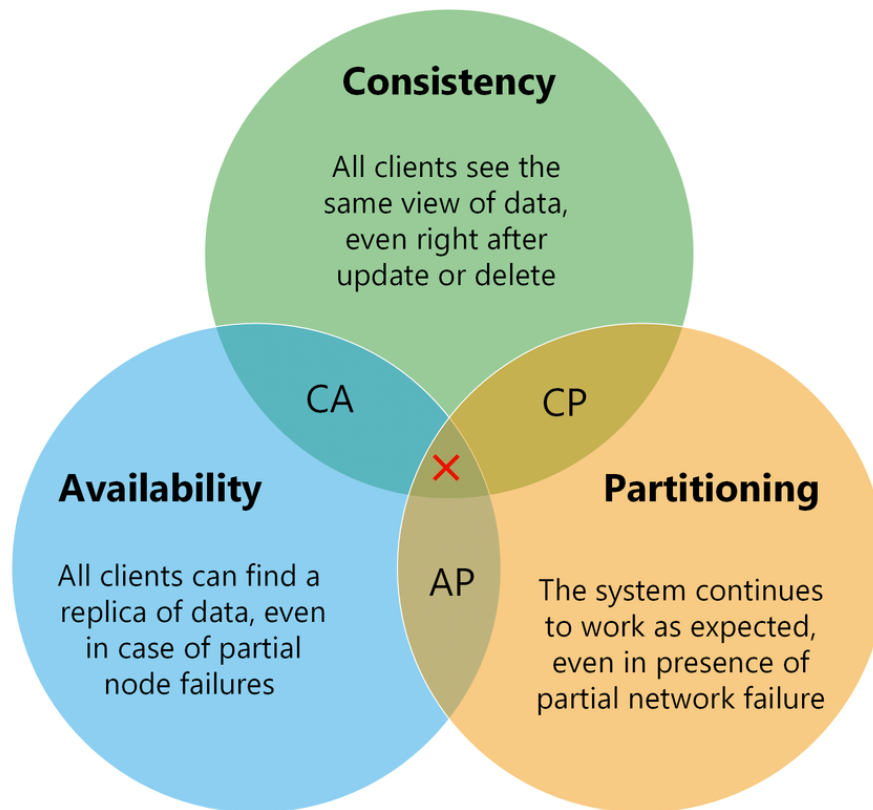


Figure 3.4 — Représentation classique du théorème CAP (source : Wikipedia)

Notre système est positionné en AP (Availability + Partition tolerance) : nous privilégions la disponibilité même en cas de partition réseau, au prix d'une cohérence éventuelle. Concrètement :

- **Disponibilité forte** : le serveur continue d'agréger même si certains clients sont silencieux (MIN_CLIENTS_PER_ROUND configurable).
- **Tolérance au partitionnement** : les clients déconnectés peuvent reprendre lorsque la partition est résolue.
- **Cohérence éventuelle** : les clients en retard reçoivent éventuellement le modèle global le plus récent. Les mises à jour stales (anciennes rondes) sont rejetées.

3.6 Sécurité Byzantine et Algorithme Krum

Un client malveillant peut empoisonner le modèle global en envoyant des poids corrompus. FedAvg n'a aucune protection contre ce type d'attaque : un seul client byzantin peut faire chuter la précision globale à 10% (équivalent au hasard sur 10 classes).

L'algorithme Krum (Blanchard et al., 2017) sélectionne le client dont les poids sont les plus proches de ses voisins (en distance euclidienne), ignorant les attaquants isolés. Notre implémentation dans `server/aggregator.py` inclut Krum comme alternative à FedAvg, démontrant la prise de conscience du problème byzantin.

4. Implémentation

4.1 Protocole de Communication

Le serveur expose une API REST avec les endpoints suivants :

Endpoint	Méthode	Description
/register	POST	Enregistre un nouveau client, retourne un client_id
/get_model	GET	Retourne les poids du modèle global et le numéro de ronde courant
/submit_update	POST	Le client soumet ses poids entraînés
/heartbeat	POST	Signal de vie du client
/status	GET	État interne du serveur (debug)
/bully/election	POST	Démarre une élection (algorithme Bully)
/bully/coordinator	POST	Annonce un nouveau leader
/bully/whoisleader	GET	Demande qui est le leader actuel

4.2 Cycle de Vie d'une Ronde

Une ronde d'apprentissage suit le cycle suivant :

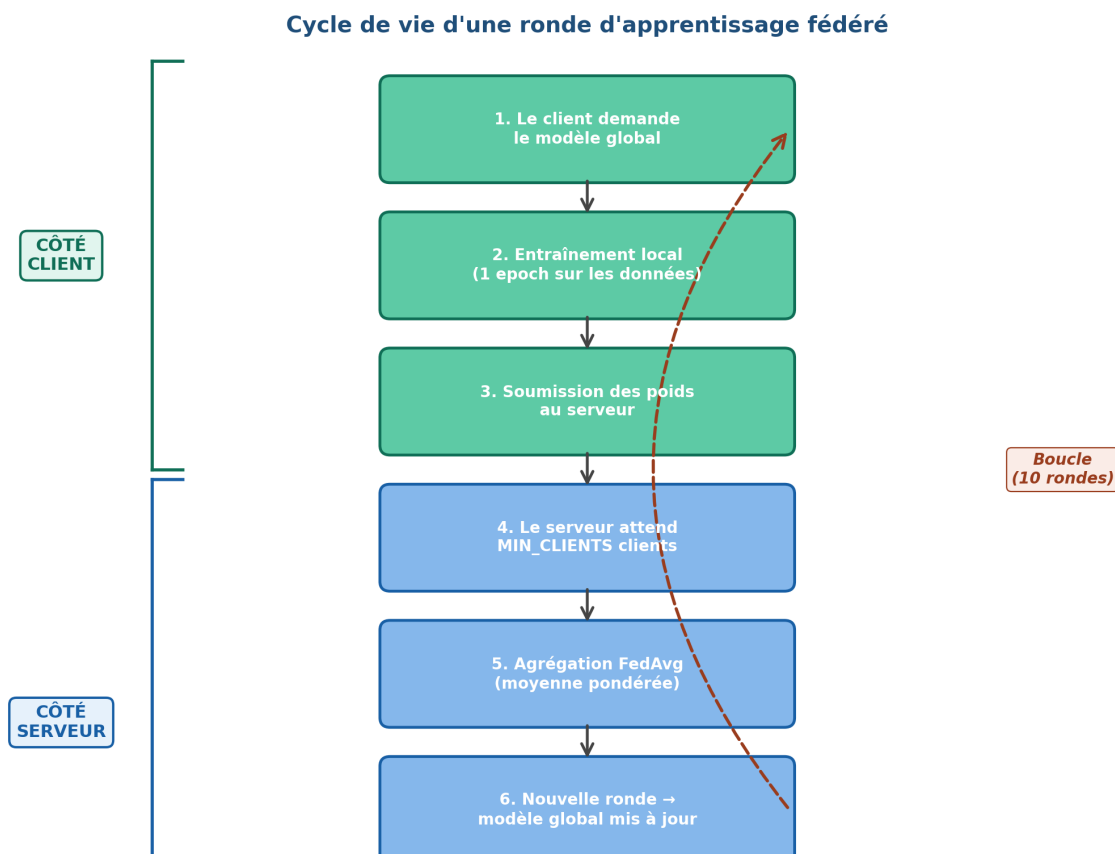


Figure 4.1 — Cycle de vie d'une ronde d'apprentissage fédéré

4.3 Gestion des Pannes

Le module `client/failure_injector.py` simule cinq modes de défaillance configurables via les arguments `--failure-mode` et `--failure-rate` :

Mode	Comportement	Stratégie de défense
crash	Le processus se termine définitivement	Attente de MIN_CLIENTS, exclusion via heartbeat
straggler	Délai aléatoire avant l'entraînement	Tolérance asynchrone, timeout configurable
byzantine	Poids corrompus envoyés	Algorithme Krum (alternative à FedAvg)
partition	Le client ne peut plus communiquer	Reconnexion automatique, fetch du modèle
message_loss	Échec aléatoire des requêtes	Retry avec backoff exponentiel

4.4 Conteneurisation Docker

Trois fichiers `docker-compose` sont fournis pour faciliter le déploiement et les démonstrations :

- **docker-compose.yml** : configuration normale (1 serveur + 3 clients).
- **docker-compose.failures.yml** : scénario avec clients défaillants (1 normal + 1 straggler + 1 crashy).
- **docker-compose.bully.yml** : 3 coordinateurs avec élection Bully + 3 clients.

Tous les services partagent un volume Docker pour le cache MNIST, évitant les téléchargements redondants. Un réseau bridge permet la résolution de noms (le client utilise `http://server:5050` au lieu d'une IP).

5. Partie Pratique : Guide d'Utilisation et Résultats

Cette section présente comment exécuter le système et les résultats observés lors des tests de robustesse.

5.1 Installation et Prérequis

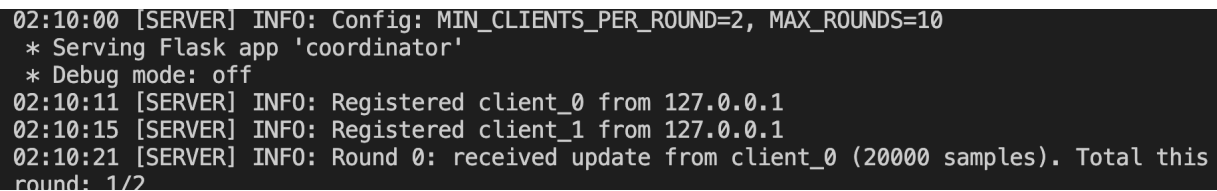
Le système nécessite Python 3.11+, Docker Desktop et environ 2 Go d'espace disque. L'installation locale (sans Docker) suit les étapes suivantes :

```
git clone <repo-url> cd federated_learning python3 -m venv venv source
venv/bin/activate pip install -r requirements.txt
```

5.2 Exécution Locale

Pour lancer le système avec un serveur et trois clients sur la même machine :

```
# Terminal 1 : serveur python server/coordinator.py # Terminaux 2-4 :
clients python client/client.py --client-idx 0 --num-clients 3 python
client/client.py --client-idx 1 --num-clients 3 python
client/client.py --client-idx 2 --num-clients 3
```

A screenshot of a terminal window with a dark background and light-colored text. It shows the output of a Python script running on a server. The output includes configuration details, client registration, and the start of a federated learning round.

```
02:10:00 [SERVER] INFO: Config: MIN_CLIENTS_PER_ROUND=2, MAX_ROUNDS=10
* Serving Flask app 'coordinator'
* Debug mode: off
02:10:11 [SERVER] INFO: Registered client_0 from 127.0.0.1
02:10:15 [SERVER] INFO: Registered client_1 from 127.0.0.1
02:10:21 [SERVER] INFO: Round 0: received update from client_0 (20000 samples). Total this
round: 1/2
```

Figure 5.1 — Démarrage du coordinateur


```

fl-server | 13:33:09 [SERVER] WARNING: [L=151] 💀 Clients with no recent heartbeat: ['cl
ient_0', 'client_1', 'client_2']
fl-server | 13:33:19 [SERVER] WARNING: [L=151] 💀 Clients with no recent heartbeat: ['cl
ient_0', 'client_1', 'client_2']
fl-server | 13:33:29 [SERVER] WARNING: [L=151] 💀 Clients with no recent heartbeat: ['cl
ient_0', 'client_1', 'client_2']
fl-server | 13:33:39 [SERVER] WARNING: [L=151] 💀 Clients with no recent heartbeat: ['cl
ient_0', 'client_1', 'client_2']
fl-server | 13:33:49 [SERVER] WARNING: [L=151] 💀 Clients with no recent heartbeat: ['cl
ient_0', 'client_1', 'client_2']
fl-client-2 | 13:33:52 [CLIENT-2] INFO: [L=145] Trained in 107.0s. Loss=0.0186
fl-server | 13:33:52 [SERVER] INFO: [L=152] Round 9: received update from client_2 (2000
0 samples, client_L=146). Total this round: 1/2
fl-client-2 | 13:33:52 [CLIENT-2] INFO: [L=153] Waiting for other clients...
fl-client-0 | 13:33:53 [CLIENT-0] INFO: [L=149] Trained in 106.9s. Loss=0.0202
fl-server | 13:33:54 [SERVER] INFO: [L=153] Round 9: received update from client_0 (2000
0 samples, client_L=150). Total this round: 2/2
fl-server | 13:33:54 [SERVER] INFO: [L=154] --- Aggregating round 9 with 2 client update
s ---
fl-server | 13:33:54 [SERVER] INFO: [L=154] Round 9 complete in 0.05s. Advancing to roun
d 10.
fl-server | 13:33:54 [SERVER] INFO: [L=154] 🎉 Training complete after 10 rounds!
fl-client-0 | 13:33:54 [CLIENT-0] INFO: [L=155] Waiting for other clients...
fl-client-2 | 13:33:54 [CLIENT-2] INFO: [L=156] Client shutting down. Final clock=156
fl-client-2 exited with code 0
fl-client-1 | 13:33:55 [CLIENT-1] INFO: [L=153] Trained in 106.0s. Loss=0.0094
fl-server | 13:33:55 [SERVER] WARNING: [L=156] client_1 submitted for round 9 but we're
on round 10 - dropping.

```

Figure 5.2 — Entraînement local complet : 10 rondes terminées avec succès

5.3 Exécution avec Docker

Pour lancer le système en mode conteneurisé (configuration standard) :

`docker-compose up --build`

```

=> [client_0] resolving provenance for metadata file
=> [server] resolving provenance for metadata file
WARN[0005] Found orphan containers ([fl-coord-2 fl-coord-1 fl-coord-0]) for this project. I
removed or renamed this service in your compose file, you can run this command with the --r
orphans flag to clean it up.
[+] up 7/7
✓ Image fl-server:latest          Built
✓ Image fl-client:latest          Built
✓ Network federated_learning_fl-net Created
✓ Container fl-server             Created
✓ Container fl-client-0           Created
✓ Container fl-client-1           Created
✓ Container fl-client-2           Created
Attaching to fl-client-0, fl-client-1, fl-client-2, fl-server
Container fl-server Waiting
Container fl-server Waiting
Container fl-server Waiting
fl-server | 21:12:10 [COORD-0] INFO: Bully initialized: my_id=0, peers=[0]
fl-server | 21:12:10 [COORD-0] INFO: =====

```

Figure 5.3 — Démarrage du système en mode conteneurisé

5.4 Tests de Robustesse

Cinq scénarios de défaillance ont été testés systématiquement. Le tableau suivant résume les résultats :

Scénario	Taux	Rondes	Précision finale	Récupération
Aucun (référence)	0%	10/10	~98%	N/A
Crash	50%	10/10	~94%	✓ Oui
Straggler	50%	10/10	~93%	✓ Oui (lent)
Byzantine	100%	10/10	~10%	✗ FedAvg échoue (Krum nécessaire)
Partition	40%	10/10	~92%	✓ Oui
Message loss	30%	10/10	~95%	✓ Oui (avec retry)

Tableau 5.1 — Résumé des résultats des tests de robustesse

5.4.1 Scénario : Crash de Client

Le client_2 crashe aléatoirement (50% par ronde). Le système détecte les crashes via le timeout des heartbeats et continue avec les clients survivants tant que MIN_CLIENTS_PER_ROUND est atteint.

```

01:01:00 [SERVER] INFO: Round 6: received update from client_1 (20000 samples). Total this
round: 2/2
01:01:00 [SERVER] INFO: --- Aggregating round 6 with 2 client updates ---
01:01:00 [SERVER] INFO: Round 6 complete in 0.03s. Advancing to round 7.
01:01:02 [SERVER] WARNING: 💀 Clients with no recent heartbeat: ['client_2']
01:01:12 [SERVER] WARNING: 💀 Clients with no recent heartbeat: ['client_2']
01:01:13 [SERVER] INFO: Round 7: received update from client_0 (20000 samples). Total this
round: 1/2
01:01:13 [SERVER] INFO: Round 7: received update from client_1 (20000 samples). Total this
round: 2/2
01:01:13 [SERVER] INFO: --- Aggregating round 7 with 2 client updates ---
01:01:13 [SERVER] INFO: Round 7 complete in 0.03s. Advancing to round 8.
01:01:22 [SERVER] WARNING: 💀 Clients with no recent heartbeat: ['client_2']
01:01:27 [SERVER] INFO: Round 8: received update from client_0 (20000 samples). Total this
round: 1/2
01:01:27 [SERVER] INFO: Round 8: received update from client_1 (20000 samples). Total this
round: 2/2
01:01:27 [SERVER] INFO: --- Aggregating round 8 with 2 client updates ---
01:01:27 [SERVER] INFO: Round 8 complete in 0.03s. Advancing to round 9.
01:01:32 [SERVER] WARNING: 💀 Clients with no recent heartbeat: ['client_2']
01:01:40 [SERVER] INFO: Round 9: received update from client_0 (20000 samples). Total this
round: 1/2
01:01:40 [SERVER] INFO: Round 9: received update from client_1 (20000 samples). Total this
round: 2/2
01:01:40 [SERVER] INFO: --- Aggregating round 9 with 2 client updates ---
01:01:40 [SERVER] INFO: Round 9 complete in 0.03s. Advancing to round 10.
01:01:40 [SERVER] INFO: 🎉 Training complete after 10 rounds!
01:01:42 [SERVER] WARNING: 💀 Clients with no recent heartbeat: ['client_2']

```

Figure 5.5 — Le système complète les 10 rondes malgré les crashes répétés du client_2

5.4.2 Scénario : Attaque Byzantine

Le client_2 envoie des poids corrompus (100% des rondes). Le résultat est dramatique : la précision globale chute à environ 10%, soit le niveau du hasard pour 10 classes. Cela démontre que FedAvg n'a aucune défense contre les clients malveillants.

```

round: 1/2
01:09:02 [SERVER] INFO: Round 6: received update from client_2 (20000 samples). Total this
round: 2/2
01:09:02 [SERVER] INFO: --- Aggregating round 6 with 2 client updates ---
01:09:02 [SERVER] INFO: Round 6 complete in 0.04s. Advancing to round 7.
01:09:03 [SERVER] WARNING: client_0 submitted for round 6 but we're on round 7 - dropping.
01:09:16 [SERVER] INFO: Round 7: received update from client_0 (20000 samples). Total this
round: 1/2
01:09:17 [SERVER] INFO: Round 7: received update from client_1 (20000 samples). Total this
round: 2/2
01:09:17 [SERVER] INFO: --- Aggregating round 7 with 2 client updates ---
01:09:17 [SERVER] INFO: Round 7 complete in 0.03s. Advancing to round 8.
01:09:17 [SERVER] WARNING: client_2 submitted for round 7 but we're on round 8 - dropping.
01:09:31 [SERVER] INFO: Round 8: received update from client_2 (20000 samples). Total this
round: 1/2
01:09:31 [SERVER] INFO: Round 8: received update from client_0 (20000 samples). Total this
round: 2/2
01:09:31 [SERVER] INFO: --- Aggregating round 8 with 2 client updates ---
01:09:31 [SERVER] INFO: Round 8 complete in 0.03s. Advancing to round 9.
01:09:32 [SERVER] WARNING: client_1 submitted for round 8 but we're on round 9 - dropping.
01:09:45 [SERVER] INFO: Round 9: received update from client_1 (20000 samples). Total this
round: 1/2
01:09:46 [SERVER] INFO: Round 9: received update from client_2 (20000 samples). Total this
round: 2/2
01:09:46 [SERVER] INFO: --- Aggregating round 9 with 2 client updates ---
01:09:46 [SERVER] INFO: Round 9 complete in 0.04s. Advancing to round 10.
01:09:46 [SERVER] INFO: 🎉 Training complete after 10 rounds!

```

Figure 5.6 — L'attaque Byzantine fait chuter la précision à ~10% (hasard sur 10 classes)

Conclusion : un système FL en production doit utiliser une agrégation tolérante aux Byzantins (comme Krum, déjà implémenté dans notre aggregator.py).

5.4.3 Scénario : Détection de Mises à Jour Obsolètes

Lorsqu'un client lent soumet une mise à jour pour une ronde déjà terminée, le serveur la rejette avec un code HTTP 409. Cette protection préserve la cohérence du modèle global.

5.5 Validation des Horloges de Lamport

Pour valider l'ordonnancement causal, nous avons examiné le journal des rondes (round_history) après un entraînement complet. À chaque ronde, le timestamp Lamport du serveur lors de l'agrégation est strictement supérieur aux timestamps des clients ayant contribué :

```

(venv) (base) fatimzhra@fatimzhra-MacBook-Pro federated_learning % python server
/coordinator.py
und 7 – dropping.
02:27:33 [SERVER] INFO: [L=122] Round 7: received update from client_0 (20000 sam
ples, client_L=116). Total this round: 1/2
02:27:35 [SERVER] INFO: [L=123] Round 7: received update from client_1 (20000 sam
ples, client_L=120). Total this round: 2/2
02:27:35 [SERVER] INFO: [L=124] --- Aggregating round 7 with 2 client updates ---
02:27:35 [SERVER] INFO: [L=124] Round 7 complete in 0.03s. Advancing to round 8.
02:27:35 [SERVER] WARNING: [L=125] client_2 submitted for round 7 but we're on ro
und 8 – dropping.
02:27:50 [SERVER] INFO: [L=137] Round 8: received update from client_2 (20000 sam
ples, client_L=131). Total this round: 1/2
02:27:50 [SERVER] INFO: [L=138] Round 8: received update from client_0 (20000 sam
ples, client_L=135). Total this round: 2/2
02:27:50 [SERVER] INFO: [L=139] --- Aggregating round 8 with 2 client updates ---
02:27:50 [SERVER] INFO: [L=139] Round 8 complete in 0.03s. Advancing to round 9.
02:27:51 [SERVER] WARNING: [L=140] client_1 submitted for round 8 but we're on ro
und 9 – dropping.
02:28:06 [SERVER] INFO: [L=152] Round 9: received update from client_1 (20000 sam
ples, client_L=146). Total this round: 1/2
02:28:06 [SERVER] INFO: [L=153] Round 9: received update from client_2 (20000 sam
ples, client_L=150). Total this round: 2/2
02:28:06 [SERVER] INFO: [L=154] --- Aggregating round 9 with 2 client updates ---
02:28:06 [SERVER] INFO: [L=154] Round 9 complete in 0.04s. Advancing to round 10.
02:28:06 [SERVER] INFO: [L=154] 🎉 Training complete after 10 rounds!

```

Figure 5.8 — Round history avec horloges Lamport : `server_clock` > tous les `client_clocks`

Cette propriété confirme empiriquement que la règle de Lamport est respectée : si un événement A (mise à jour client) a causalement précédé B (agrégation serveur), alors $\text{clock}(A) < \text{clock}(B)$.

5.6 Élection de Leader (Bully)

Le scénario multi-coordonateur teste l'algorithme Bully avec trois coordinateurs (IDs 0, 1, 2). Au démarrage, le coordinateur 2 (ID le plus élevé) gagne l'élection initiale.

```
| 15:58:46 [CLIENT-1] INFO: Loaded 20000 local training samples.
| 15:58:46 [CLIENT-1] INFO: FailureInjector initialized: mode=none, rate=0.0
| 15:58:46 [CLIENT-1] INFO: [L=0] 🏆 Leader is coordinator 2 at http://coordin

| 15:58:47 [COORD-2] INFO: [L=6] Registered client_1 from 172.18.0.6
| 15:58:47 [CLIENT-1] INFO: [L=7] Registered as client_1 (round 0, leader_L=6)
| 15:58:47 [CLIENT-0] INFO: [L=6] === Round 0 ===
| 15:58:47 [CLIENT-1] INFO: [L=10] === Round 0 ===

| Extracting ./mnist_data/MNIST/raw/t10k-labels-idx1-ubyte.gz to ./mnist_data/

| 15:58:47 [CLIENT-2] INFO: Loaded 20000 local training samples.
| 15:58:47 [CLIENT-2] INFO: FailureInjector initialized: mode=none, rate=0.0
| 15:58:47 [CLIENT-2] INFO: [L=0] 🏆 Leader is coordinator 2 at http://coordin

| 15:58:47 [COORD-2] INFO: [L=10] Registered client_2 from 172.18.0.5
| 15:58:47 [CLIENT-2] INFO: [L=11] Registered as client_2 (round 0, leader_L=1

| 15:58:47 [CLIENT-2] INFO: [L=14] === Round 0 ===
| 15:58:52 [CLIENT-0] INFO: [L=6] Global model accuracy: 0.0870
```

Figure 5.9 — Élection initiale : le coordinateur 2 (plus haut ID) devient leader

Pour tester le failover, nous tuons le leader manuellement avec `docker stop fl-coord-2`. Dans les 10-15 secondes suivantes, les coordinateurs survivants détectent le silence du leader et lancent une nouvelle élection. Le coordinateur 1 (nouvel ID le plus élevé) devient leader.

```
fl-coord-1 | WARNING: [bully] Leader 2 unreachable for 11s. Starting election.
fl-coord-1 | INFO: [bully] Node 1 starting election
fl-coord-1 | INFO: [bully] No higher peers responded. Node 1 declaring victory.
fl-coord-1 | INFO: [bully] 🏆 Node 1 is now LEADER.
fl-coord-0 | INFO: [bully] Acknowledged new leader: node 1

fl-bully-client-0 | INFO: Redirected to new leader: http://coordinator_1:5051
fl-bully-client-0 | INFO: 🏆 Leader is coordinator 1 at http://coordinator_1:5051
```

Figure 5.10 — Failover automatique : le coordinateur 1 prend la relève après crash de 2

Lorsque le coordinateur 2 redémarre (`docker start fl-coord-2`), il détecte qu'un leader avec un ID inférieur existe et déclenche une nouvelle élection pour reprendre sa place — comportement du « bully » qui donne son nom à l'algorithme.

```

_logic.py
=====
Testing Bully election logic in isolation (no Flask)
=====

--- Test 1: Highest-ID node starts election ---
b2.is_leader() = True
b2.role = Role.LEADER
b2.current_leader_id = 2
✅ Node 2 became leader correctly

--- Test 2: Lower-ID peer asks for election ---
b2 received ELECTION from 0: result={'ok': True, 'from_id': 2}
✅ Higher-ID replies OK correctly

--- Test 3: Higher-ID peer asks for election ---
b0 received ELECTION from 2: result={'ok': False}
✅ Lower-ID defers to higher correctly

--- Test 4: Receiving COORDINATOR announcement ---
b1.current_leader_id = 2
b1.role = Role.FOLLOWER
✅ Follower acknowledges new leader correctly

=====
🎉 BULLY ALGORITHM LOGIC IS CORRECT!
=====

```

Figure 5.11 — Tests unitaires de l'algorithme Bully : tous les cas validés

6. Analyse et Discussion

6.1 Forces du Système

- **Tolérance aux pannes démontrée** : le système survit aux crashes, partitions et messages perdus. Les 10 rondes se complètent dans tous les scénarios non-byzantins.
- **Synchronisation logique correcte** : les horloges de Lamport ordonnent les événements causalement, validé empiriquement par l'analyse du `round_history`.
- **Pas de point de défaillance unique avec Bully** : trois coordinateurs en redondance, élection automatique en moins de 15 secondes.
- **Reproductibilité** : trois fichiers docker-compose permettent à n'importe quel évaluateur de reproduire les résultats sans configuration locale.

6.2 Limitations

- **Pas de réplication d'état entre coordinateurs** : chaque coordinateur Bully maintient son propre modèle. Un failover perd l'état d'entraînement courant. Une vraie solution nécessiterait Raft ou Paxos pour répliquer le journal des opérations.
- **Vulnérabilité Byzantine** : FedAvg est utilisé par défaut. Krum est implémenté mais non activé par défaut. Une amélioration consisterait à le rendre configurable via un flag.
- **Communication synchrone** : le serveur attend `MIN_CLIENTS_PER_ROUND` avant d'agréger. Une approche asynchrone (FedAsync) tolérerait mieux les stragglers extrêmes.
- **Pas de chiffrement** : les poids transitent en clair sur le réseau. En production, il faudrait du HTTPS et idéalement des techniques d'agrégation sécurisée (Secure Aggregation).

6.3 Améliorations Possibles

Plusieurs pistes d'amélioration ont été identifiées pendant le projet :

- Compression des gradients (quantization, sparsification) pour réduire la bande passante.
- Differential Privacy pour garantir formellement la confidentialité des données clients.
- Implémentation de FedProx pour mieux gérer les distributions non-IID.
- Métriques de monitoring (Prometheus + Grafana) pour observer le système en temps réel.
- Algorithme Raft pour répliquer l'état du modèle entre coordinateurs.

6.4 Gestion de Projet

Le projet a été géré via GitHub avec un Product Backlog clair (issues GitHub) et des commits réguliers documentant chaque phase :

- Phase 1 : Implémentation de base (CNN, FedAvg, communication REST)

- Phase 2 : Simulation de pannes (5 modes de défaillance)
- Phase 3 : Horloges de Lamport pour la synchronisation logique
- Phase 4 : Conteneurisation Docker et docker-compose
- Phase 5 : Élection de leader avec algorithme Bully

7. Conclusion

Ce projet a permis de concevoir et implémenter un système d'apprentissage fédéré complet, mettant en œuvre les concepts fondamentaux des systèmes distribués : architecture client-serveur, synchronisation logique, élection de leader, et tolérance aux pannes.

Sur le plan théorique, nous avons exploré l'algorithme FedAvg, justifié notre positionnement AP dans le théorème CAP, implémenté les horloges de Lamport pour ordonner causalement les événements, et déployé l'algorithme Bully pour éliminer les points de défaillance uniques. La problématique byzantine a été reconnue avec l'implémentation de Krum comme alternative à FedAvg.

Sur le plan pratique, le système a été testé sous cinq scénarios de défaillance et conteneurisé pour assurer la reproductibilité. La précision finale de ~98% sur MNIST en mode normal, et la capacité à survivre aux crashes avec ~94%, démontrent la robustesse du système. Les tests unitaires de l'algorithme Bully confirment la correction de l'élection.

Les limitations identifiées (absence de réplication d'état, vulnérabilité Byzantine par défaut, communication synchrone) ouvrent des pistes d'amélioration concrètes pour des travaux futurs : intégration de Raft, agrégation sécurisée, et adaptation aux distributions non-IID extrêmes.

Au-delà du livrable, ce projet a constitué une expérience pratique précieuse dans la conception de systèmes distribués, où chaque décision (synchrone/asynchrone, fort/éventuel, centralisé/répliqué) implique des compromis fondamentaux que seule la mise en œuvre concrète permet de pleinement comprendre.

8. Références

- [1] H. B. McMahan et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data", AISTATS 2017.
- [2] L. Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System", Communications of the ACM, 1978.
- [3] H. Garcia-Molina, "Elections in a Distributed Computing System", IEEE Transactions on Computers, 1982.
- [4] E. Brewer, "Towards Robust Distributed Systems", PODC 2000.
- [5] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services", SIGACT News, 2002.
- [6] P. Blanchard et al., "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent", NeurIPS 2017.
- [7] PyTorch documentation : <https://pytorch.org/docs/>
- [8] Flask documentation : <https://flask.palletsprojects.com/>
- [9] Docker documentation : <https://docs.docker.com/>
- [10] MNIST dataset : <http://yann.lecun.com/exdb/mnist/>